Análisis de Gentrificación — CDMX

Adaptación de `batch_analysis.ipynb` (Mauro et al., 2025) para Ciudad de México.

Diferencias clave respecto al paper original:

Parámetro	Paper USA	CDMX
Grid	7×7 = 49 celdas	5×5 = 25 celdas
num_agents	2^7–2^13	4096 (2^12)
Distribución ingresos	USA Census	ENIGH 2022 CDMX
Cutoff L/M/H	~38/57/5%	38.3/56.7/5.0%
Reps	150	150

Ejecutar desde: `~/Gentrificacion_CDMX/Gentrification/CDMX/`

0. Imports y configuración

In [1]:
```
!uv pip install colorcet
```

```
Using Python 3.11.15 environment at: /Users/mateos_kvn/Gentrificacion_CDMX/G
entrification/.venv
Checked 1 package in 2ms
```

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from collections import defaultdict
from tqdm.auto import tqdm
from utils_gent_measure import *
import os

# Importar utils desde la carpeta CDMX
from ABM_MeanField_Cells.utils import *

np.set_printoptions(linewidth=400)

%matplotlib inline

sns.set()
sns.set(rc={'figure.figsize': (16, 9)})
sns.set_style("whitegrid")
sns.set_context("talk")

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
```

```
warnings.simplefilter(action='ignore', category=pd.errors.PerformanceWarning

# ── CDMX: grid 5×5 en lugar de 7×7 del paper original ──
width = height = 5
```

In [3]:
```
import colorcet as cc
# 25 colores para 25 celdas (5×5)
palette = sns.color_palette(cc.glasbey, n_colors=width * height)
```

In [4]:
```
# ── Variables CDMX ──────────────────────────────────────────────────
# CAMBIO: num_agents=4096 (único tamaño simulado en CDMX)
# El paper original corre 2^7 a 2^13; aquí solo tenemos 2^12
#num_agents = 2**12   # 4096

num_agents = 2**12   # tamaño principal para análisis individual
n_agents_cdmx = [1024, 2048, 4096, 8192]   # todos los tamaños para curva de

mode       = "improve"
size       = "big"
starting_deployment = f"centre_segr-{size}"   # "centre_segr-big"

# Rutas — misma estructura que el repo original
directory       = f"out/batch_results/{mode}/{starting_deployment}/{num_ager
directory_exps  = directory + "exps/"
intermediate_dir = os.path.join(directory, "intermediate")

print("exps:        ", directory_exps)
print("intermediate:", intermediate_dir)
print("exps existentes:        ", len([f for f in os.listdir(directory_exps)
print("intermediate existentes:", len([f for f in os.listdir(intermediate_di

os.makedirs(f"out/plots/{starting_deployment}/{num_agents}", exist_ok=True)
```

```
exps:         out/batch_results/improve/centre_segr-big/4096agents/exps/
intermediate: out/batch_results/improve/centre_segr-big/4096agents/intermedi
ate
exps existentes:         1500
intermediate existentes: 6750
```

1. G_bin y G_net — Boxplots por p_g

Figura equivalente a **Fig. 2** del paper (Mauro et al., 2025).
Muestra qué fracción de celdas experimenta gentrificación en función de `p_H` (probabilidad de movimiento estratégico de agentes H).

In [5]:
```
p_g_list   = [0.0, 0.01, 0.05, 0.1, 0.15]
h_list     = [20]    # ventana temporal agentes H
delta_list = [15]    # ventana de suavizado para detección de eventos

pg2frac_gamma = defaultdict(list)   # G_net por p_g
pg2frac_chi   = defaultdict(list)   # G_bin por p_g
pg2steps      = defaultdict(list)   # pasos hasta convergencia
```

```
for p_g in tqdm(p_g_list, desc="p_g"):
    p_g_str = f"pg_{p_g}_"

    for h in h_list:
        h_str = f"h_{h}_"

        for delta in delta_list:
            delta_str = f"delta_{delta}_"

            for file_name in os.listdir(intermediate_dir):
                if p_g_str in file_name and h_str in file_name and delta_str
                    file_path = os.path.join(intermediate_dir, file_name)
                    df = pd.read_csv(file_path, index_col=0)

                    # ── G_net (net_avg_prod) ──────────────────────────────
                    if "net_avg_prod" in file_name:
                        gentrified_gamma_cells = []
                        for cell in df.columns:
                            peaks = find_peaks_custom(df[cell].reset_index(d
                            if len(peaks) > 0:
                                gentrified_gamma_cells.append(cell)
                        pg2frac_gamma[p_g].append(len(gentrified_gamma_cells

                    # ── G_bin (chi_hat) ───────────────────────────────────
                    if "chi_hat" in file_name:
                        gentrified_dummy_cells = []
                        for cell in df.columns:
                            peaks = find_shifts(df[cell].reset_index(drop=Tr
                            if len(peaks) > 0:
                                gentrified_dummy_cells.append(cell)
                        pg2steps[p_g].append(len(df))
                        pg2frac_chi[p_g].append(len(gentrified_dummy_cells)

print("Réplicas por p_g (G_bin):", {p: len(v) for p, v in pg2frac_chi.items(
print("Réplicas por p_g (G_net):", {p: len(v) for p, v in pg2frac_gamma.item
```

```
p_g:   0%|          | 0/5 [00:00<?, ?it/s]
Réplicas por p_g (G_bin): {0.0: 150, 0.01: 150, 0.05: 150, 0.1: 150, 0.15: 1
50}
Réplicas por p_g (G_net): {0.0: 150, 0.01: 150, 0.05: 150, 0.1: 150, 0.15: 1
50}
```

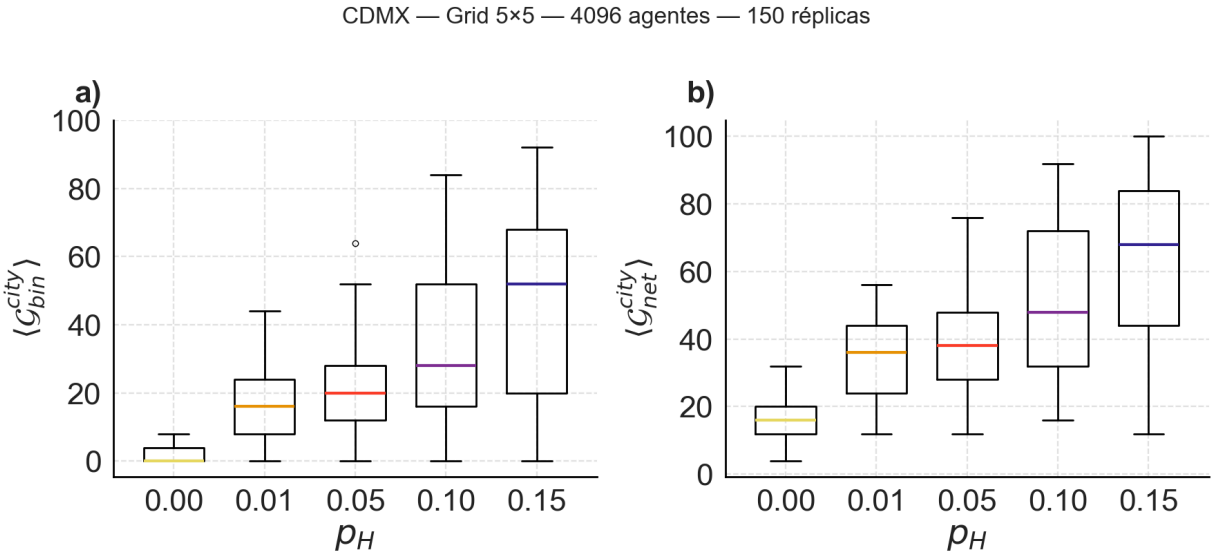
In [6]:

```
def create_boxplot(ax, data_dict, ylabel, xlabel, yticks=False, title=None):
    p_g_values = list(data_dict.keys())
    values     = list(data_dict.values())

    spacing   = 0.3
    positions = np.arange(len(p_g_values)) * spacing
    palette   = sns.color_palette("CMRmap_r", len(p_g_values))

    bp = ax.boxplot(
        values, positions=positions, widths=0.2,
        boxprops=dict(linewidth=2, zorder=2),
        whiskerprops=dict(linewidth=2),
        capprops=dict(linewidth=2)
    )
```

```
    for i, median in enumerate(bp['medians']):
        median.set_color(palette[i])
        median.set_linewidth(3)
        median.set_zorder(3)

    ax.set_xticks(positions)
    ax.set_xticklabels([f'{p:.2f}' for p in p_g_values], fontsize=32)
    ax.set_xlim(-0.2, spacing * (len(p_g_values) - 1) + 0.2)
    ax.set_xlabel(xlabel, fontsize=36)

    if yticks:
        ax.set_yticks(np.linspace(0, 1, 6))
        ax.set_yticklabels([f'{int(i * 100)}' for i in np.linspace(0, 1, 6)]
        ax.set_ylabel(ylabel, fontsize=32)
    else:
        ax.set_yticks(np.linspace(0, 300, 7))
        ax.set_yticklabels([f'{int(i)}' for i in np.linspace(0, 300, 7)], fo
        ax.set_ylabel(ylabel, fontsize=32)

    ax.grid(alpha=0.5, linestyle='--')
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    ax.spines['left'].set_linewidth(2)
    ax.spines['bottom'].set_linewidth(2)
    ax.tick_params(width=2)
    ax.spines['bottom'].set_color('black')
    ax.spines['left'].set_color('black')

    if title:
        ax.text(-0.08, 1.05, title, fontsize=32, fontweight='bold', transfor
```

In [7]:

```
# ── Figura equivalente a Fig. 2 del paper — CDMX ──────────────────────
fig, axs = plt.subplots(1, 2, figsize=(18, 8))

create_boxplot(axs[0], pg2frac_chi,   r'$ \langle \mathcal{G}_{bin}^{city} \
create_boxplot(axs[1], pg2frac_gamma, r'$ \langle \mathcal{G}_{net}^{city} \

plt.suptitle("CDMX — Grid 5×5 — 4096 agentes — 150 réplicas", fontsize=24, y
plt.tight_layout()
plt.savefig(f"out/plots/{starting_deployment}/{num_agents}/cdmx_boxplot_Gbir
plt.show()
```



2. Sección "Changing density" — DESACTIVADA

> **Nota CDMX:** El paper original barre `num_agents` de 2^7 a 2^13.
> Para CDMX solo se corrió **num_agents = 4096 (2^12)**, por lo que esta sección no aplica.
> Si en el futuro se corren otros tamaños, reactivar la celda quitando `%%script false`.

In [ ]:

```
%%script false --no-raise-error
# ── DESACTIVADO: solo disponible si se corren múltiples num_agents ──────────
# Para reactivar: quitar la línea %%script false y correr con:
#   [2**x for x in range(7, 13)]  →  [128, 256, 512, 1024, 2048, 4096]

p_g_list   = [0.0, 0.01, 0.05, 0.1, 0.15]
h_list     = [20]
delta_list = [15]

nagents2frac_gamma = defaultdict(lambda: defaultdict(list))
nagents2frac_chi   = defaultdict(lambda: defaultdict(list))
pg2steps_multi     = defaultdict(lambda: defaultdict(list))

for n_agents in tqdm([2**x for x in range(7, 13)]):
    inter = f"out/batch_results/{mode}/centre_segr-{size}/{n_agents}agents/i
    for p_g in p_g_list:
        p_g_str   = f"pg_{p_g}_"
        for h in h_list:
            h_str = f"h_{h}_"
            for delta in delta_list:
                delta_str = f"delta_{delta}_"
                for file_name in os.listdir(inter):
                    if p_g_str in file_name and h_str in file_name and delta
                        file_path = os.path.join(inter, file_name)
                        df = pd.read_csv(file_path, index_col=0)
                        if "net_avg_prod" in file_name:
                            cells = [c for c in df.columns if len(find_peaks
```

```
                            nagents2frac_gamma[n_agents][p_g].append(len(cel
                        if "chi_hat" in file_name:
                            cells = [c for c in df.columns if len(find_shift
                            pg2steps_multi[n_agents][p_g].append(len(df))
                            nagents2frac_chi[n_agents][p_g].append(len(cells
```

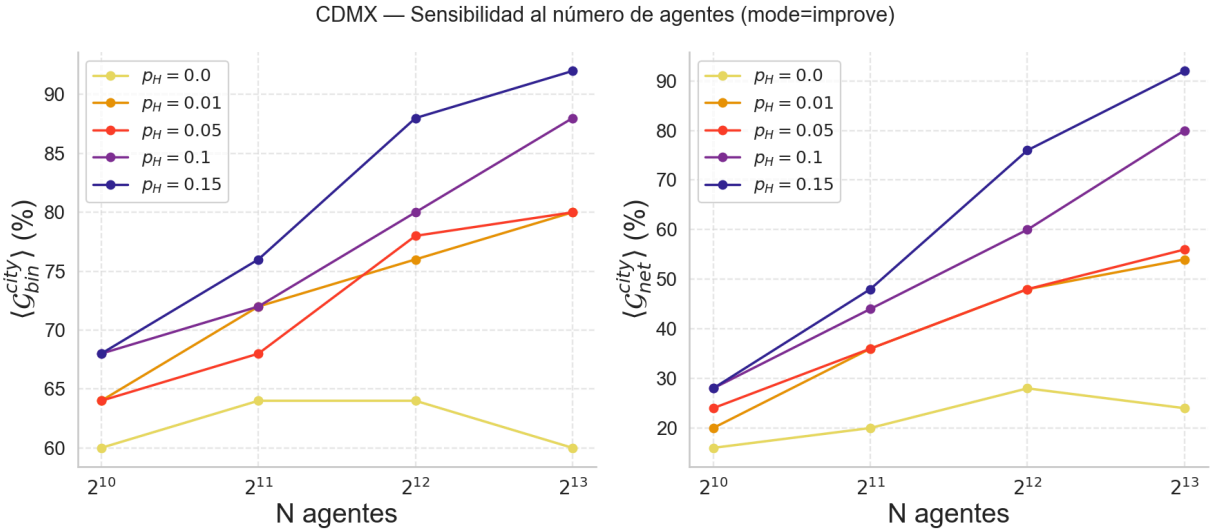
In [9]:

```
# ── Sección "Changing density" — CDMX activa ──────────────────────────
p_g_list   = [0.0, 0.01, 0.05, 0.1, 0.15]
h_list     = [20]
delta_list = [15]

nagents2frac_gamma = defaultdict(lambda: defaultdict(list))
nagents2frac_chi   = defaultdict(lambda: defaultdict(list))
pg2steps_multi     = defaultdict(lambda: defaultdict(list))

for n_ag in tqdm(n_agents_cdmx, desc="N agentes"):
    inter = f"out/batch_results/{mode}/centre_segr-{size}/{n_ag}agents/inter
    if not os.path.exists(inter):
        print(f"⚠ No existe: {inter} — omitiendo")
        continue

    for p_g in p_g_list:
        p_g_str = f"pg_{p_g}_"
        for h in h_list:
            h_str = f"h_{h}_"
            for delta in delta_list:
                delta_str = f"delta_{delta}_"

                for file_name in os.listdir(inter):
                    if p_g_str in file_name and h_str in file_name and delta
                        df = pd.read_csv(os.path.join(inter, file_name), inc

                        # FIX: eliminar columnas con todos NA antes de proce
                        df = df.dropna(axis=1, how='all')
                        if df.empty or len(df.columns) == 0:
                            continue

                        if "net_avg_prod" in file_name:
                            cells = [c for c in df.columns
                                     if len(find_peaks_custom(df[c].fillna(0
                            nagents2frac_gamma[n_ag][p_g].append(
                                len(cells) / len(df.columns))
                        if "chi_hat" in file_name:
                            cells = [c for c in df.columns
                                     if len(find_shifts(df[c].fillna(0).rese
                            pg2steps_multi[n_ag][p_g].append(len(df))
                            nagents2frac_chi[n_ag][p_g].append(
                                len(cells) / len(df.columns))

print("Tamaños procesados:", sorted(nagents2frac_gamma.keys()))
print("Réplicas por N (G_bin, p_g=0.1):",
      {n: len(nagents2frac_chi[n].get(0.1, [])) for n in n_agents_cdmx})
```

```
N agentes:   0%|          | 0/4 [00:00<?, ?it/s]
```

```
Tamaños procesados: [1024, 2048, 4096, 8192]
Réplicas por N (G_bin, p_g=0.1): {1024: 148, 2048: 150, 4096: 150, 8192: 14
9}
```

In [10]:
```python
# ── Figura: G_bin y G_net vs N para cada p_g ──────────────────────────
palette_density = sns.color_palette("CMRmap_r", len(p_g_list))
fig, axs = plt.subplots(1, 2, figsize=(18, 8))

for i, p_g in enumerate(p_g_list):
    # G_bin
    medians_chi = [np.median(nagents2frac_chi[n][p_g]) * 100
                   if nagents2frac_chi[n][p_g] else 0
                   for n in n_agents_cdmx]
    axs[0].plot(n_agents_cdmx, medians_chi, marker='o', label=f'$p_H={p_g}$'
                color=palette_density[i], linewidth=2.5)

    # G_net
    medians_gam = [np.median(nagents2frac_gamma[n][p_g]) * 100
                   if nagents2frac_gamma[n][p_g] else 0
                   for n in n_agents_cdmx]
    axs[1].plot(n_agents_cdmx, medians_gam, marker='o', label=f'$p_H={p_g}$'
                color=palette_density[i], linewidth=2.5)

for ax, title in zip(axs, [r'$\langle \mathcal{G}_{bin}^{city} \rangle$',
                           r'$\langle \mathcal{G}_{net}^{city} \rangle$'])
    ax.set_xlabel('N agentes', fontsize=28)
    ax.set_ylabel(title + ' (%)', fontsize=28)
    ax.set_xscale('log', base=2)
    ax.set_xticks(n_agents_cdmx)
    ax.set_xticklabels([f'$2^{{{int(np.log2(n))}}}$' for n in n_agents_cdmx]
    ax.tick_params(labelsize=20)
    ax.legend(fontsize=18)
    ax.grid(alpha=0.4, linestyle='--')
    sns.despine(ax=ax)

plt.suptitle("CDMX — Sensibilidad al número de agentes (mode=improve)", font
plt.tight_layout()
plt.savefig(f"out/plots/{starting_deployment}/{num_agents}/cdmx_density_curv
            bbox_inches='tight')
plt.show()
```



3. Visualización de una réplica: Riqueza mediana y G_net

Figura equivalente a **Fig. 3** del paper.
Muestra cómo evoluciona la riqueza mediana por celda y la métrica G_net para una réplica individual.

- La línea vertical en `x = h+1 = 21` marca el momento en que los agentes H comienzan a moverse estratégicamente.
- Las celdas coloreadas son las que presentaron al menos un evento de gentrificación detectado por G_net.

In [11]:
```
# ── Parámetros de la réplica a visualizar ──────────────────────────
# CAMBIO vs original: starting_deployment incluye '-big', num_agents=4096
starting_deployment_single = "centre_segr-big"   # era "centre_segr" sin '-b
num_agents_single          = 2**12               # era 2**11

# CAMBIO: directorio ahora incluye mode='improve' en la ruta
directory_single      = f"out/batch_results/{mode}/{starting_deployment_sing
intermediate_dir_single = os.path.join(directory_single, "intermediate")
exp_directory_single    = os.path.join(directory_single, "exps")

os.makedirs(f"out/plots/{starting_deployment_single}/{num_agents_single}", e

print("Ruta intermedia:", intermediate_dir_single)
print("Ruta exps:      ", exp_directory_single)
```

```
Ruta intermedia: out/batch_results/improve/centre_segr-big/4096agents/interm
ediate
Ruta exps:       out/batch_results/improve/centre_segr-big/4096agents/exps
```

In [12]:
```
# ── Seleccionar réplica ────────────────────────────────────────────
# Cambiar p_g y rep para explorar distintas corridas
p_g   = 0.1    # prueba con 0.01, 0.05, 0.1, 0.15 para ver diferencias
h     = 20
rep   = 0      # réplica 0 a 149
```

```
delta = 15

p_g_str   = f"pg_{p_g}_"
h_str     = f"h_{h}_"
rep_str   = f"rep_{rep}_"
delta_str = f"delta_{delta}_"
```

In [13]:
```
chi_hat_filename   = f"{p_g_str}{h_str}{rep_str}{delta_str}results_chi_hat.c
gamma_filename     = f"{p_g_str}{h_str}{rep_str}{delta_str}results_net_avg_p
chi_filename       = f"{p_g_str}{h_str}{rep_str}{delta_str}results_chi.csv"
exec_model_filename = f"{p_g_str}{h_str}{rep_str}results_model.csv"
exec_agents_filename = f"{p_g_str}{h_str}{rep_str}results_agents.csv"

df_chi_hat   = pd.read_csv(os.path.join(intermediate_dir_single, chi_hat_fil
df_gamma     = pd.read_csv(os.path.join(intermediate_dir_single, gamma_filer
df_chi       = pd.read_csv(os.path.join(intermediate_dir_single, chi_filenam
df_exec_model = pd.read_csv(os.path.join(exp_directory_single, exec_model_fi

df = read_df_agents(os.path.join(exp_directory_single, exec_agents_filename)

# CDMX: 25 celdas (0..24)
mass       = range(df["pos"].max()[0] + 1)
num_steps  = max(df["Step"])
all_sources = [(o, d) for o in mass for d in mass]

df_cell_median = get_median_richness_df(df_exec_model, all_sources)

print(f"Celdas en G_net: {len(df_gamma.columns)}  |  Pasos: {len(df_gamma)}"
```

```
Celdas en G_net: 25  |  Pasos: 128
```

In [14]:
```
# Columnas de df_gamma como tuplas
df_gamma.columns = [eval(x) for x in df_gamma.columns]
```

In [15]:
```
import colorcet as cc
sns.set()
sns.set(rc={'figure.figsize': (16, 9)})
sns.set_style("whitegrid")
sns.set_context("talk")
```

In [21]:
```
# ── Ventana de visualización ──────────────────────────────────────────
start = 0
end   = 81   # ajustar según num_steps de tu corrida (máximo: num_steps)

df_gamma_end = df_gamma.iloc[start:end].copy()
df_gamma_end.fillna(0, inplace=True)
# Mantener solo celdas con actividad G_net > 0
df_gamma_end = df_gamma_end.loc[:, (df_gamma_end.sum(axis=0) > 0)]

#print(f"Celdas con G_net > 0: {len(df_gamma_end.columns)} de {len(mass)}")
print(f"Celdas con G_net > 0: {len(df_gamma_end.columns)} de {len(mass)**2}"
df_gamma_end.head(3)
```

```
Celdas con G_net > 0: 12 de 25
```

Out[21]:

	(0, 0)	(0, 3)	(0, 4)	(1, 0)	(2, 1)	(2, 4)	(3, 0)	(3, 4)	(4, 0)	(4, 1)	(4, 2)	(4, 3)
1	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

In [17]:
```
palette_single = sns.color_palette(n_colors=len(df_gamma_end.columns))

# Ajuste para paletas de tamaño específico (heredado del original)
if len(palette_single) > 7:
    if len(palette_single) > 10:
        palette_single = sns.color_palette(n_colors=len(df_gamma_end.columns
        palette_single.pop(10)
        palette_single.pop(7)
    else:
        palette_single = sns.color_palette(n_colors=len(df_gamma_end.columns
        palette_single.pop(7)

print(f"Colores en paleta: {len(palette_single)}")
```

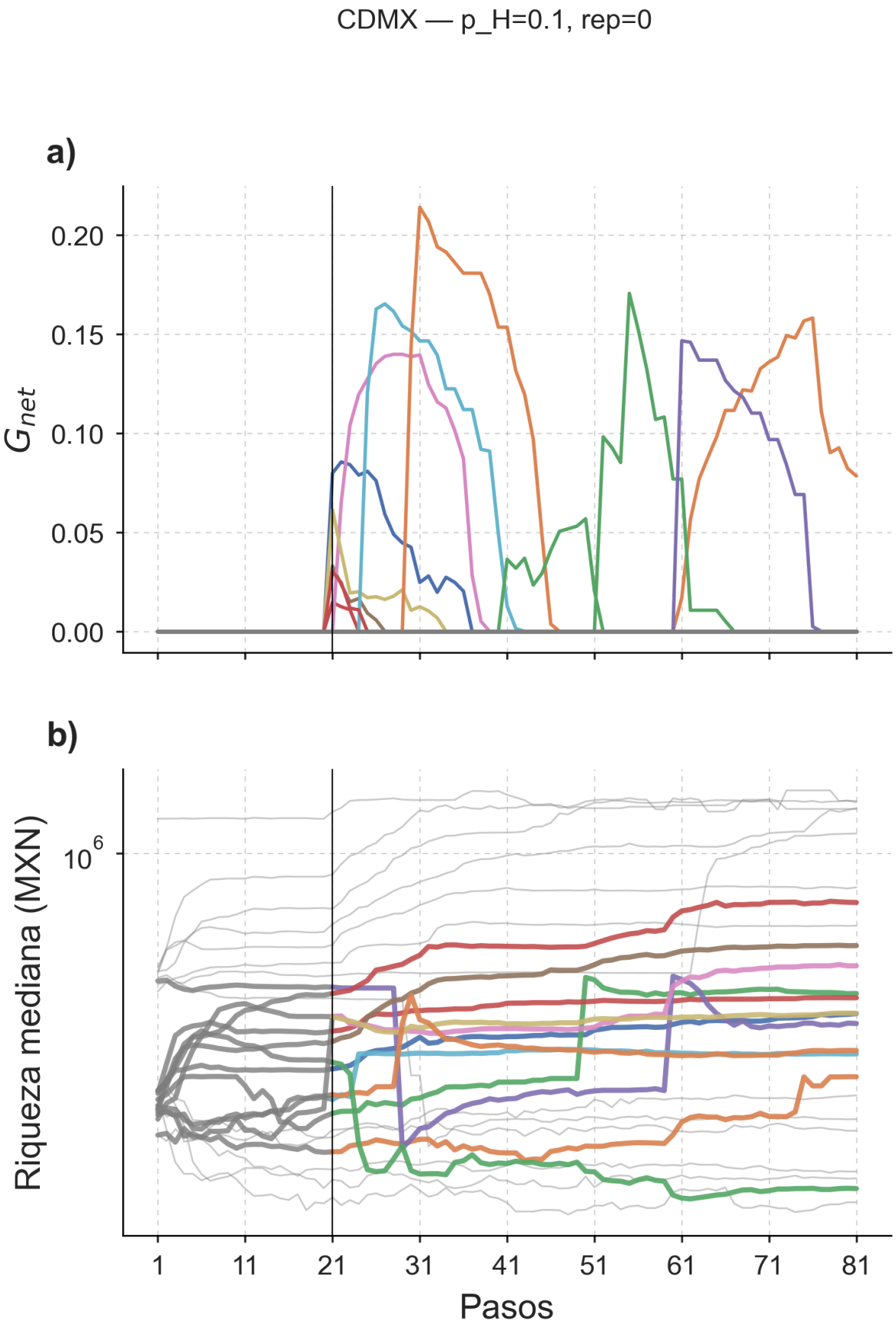
```
Colores en paleta: 12
```

In [18]:
```
df_median_end = df_cell_median.iloc[start:end]

# Separar celdas con/sin evento G_net
df_median_end_common     = df_median_end[df_gamma_end.columns]
df_median_end_not_common = df_median_end.drop(columns=df_gamma_end.columns)

num_not_common = len(df_median_end_not_common.columns)
grey_palette   = ["#808080"] * num_not_common
```

In [19]:
```
# ── CDMX: split en x=21 (h+1, momento en que H empieza a moverse) ──────
split_x = h + 1   # = 21

df_median_end_not_common_grey  = df_median_end_not_common[df_median_end_not_
df_median_end_not_common_color = df_median_end_not_common[df_median_end_not_
df_median_end_common_grey      = df_median_end_common[df_median_end_common.i
df_median_end_common_color     = df_median_end_common[df_median_end_common.i

shared_ticks = list(range(start + 1, end + 5, 10))

fig, axes = plt.subplots(2, 1, figsize=(4, 5.5), gridspec_kw={'hspace': 0.25

# ── Panel a) G_net ──────────────────────────────────────────────────────
sns.lineplot(data=df_gamma_end, palette=palette_single, dashes=False, legend
axes[0].plot([start + 1, end], [0, 0], color='grey', linewidth=1.25, linesty
axes[0].set_ylabel(r"$G_{{net}}$", fontsize=10)
axes[0].tick_params(axis='both', bottom=True, left=True, labelsize=8, length
axes[0].set_xticks(shared_ticks)
axes[0].grid(True, linestyle='dashed', dashes=(6, 6), alpha=0.7, linewidth=0
axes[0].axvline(x=split_x, color='black', linestyle='solid', linewidth=0.4)

# ── Panel b) Riqueza mediana ────────────────────────────────────────────
```

```
sns.lineplot(data=df_median_end_not_common_grey,  palette=grey_palette,   da
sns.lineplot(data=df_median_end_common_grey,      palette=grey_palette,   da
sns.lineplot(data=df_median_end_not_common_color, palette=grey_palette,   da
sns.lineplot(data=df_median_end_common_color,     palette=palette_single, da

axes[1].set_yscale("log")
axes[1].set_ylabel("Riqueza mediana (MXN)", fontsize=10)
axes[1].set_xlabel("Pasos", fontsize=10)
axes[1].tick_params(axis='both', bottom=True, left=True, labelsize=8, length
axes[1].set_xticks(shared_ticks)
axes[1].grid(True, linestyle='dashed', dashes=(6, 6), alpha=0.7, linewidth=0
axes[1].axvline(x=split_x, color='black', linestyle='solid', linewidth=0.4)

sns.despine()
for ax in axes:
    ax.spines['bottom'].set_color('black')
    ax.spines['left'].set_color('black')
    ax.spines['bottom'].set_linewidth(0.5)
    ax.spines['left'].set_linewidth(0.5)

axes[0].text(-0.1, 1.05, "a)", fontsize=10, fontweight='bold', transform=axe
axes[1].text(-0.1, 1.05, "b)", fontsize=10, fontweight='bold', transform=axe

plt.suptitle(f"CDMX — p_H={p_g}, rep={rep}", fontsize=8, y=1.01)
plt.tight_layout()
plt.savefig(f"out/plots/{starting_deployment_single}/{num_agents_single}/cdm
plt.show()
```

```
/var/folders/rw/j2_s688s5x9f9k_xg2xhfq980000gn/T/ipykernel_3008/2085485653.p
y:24: UserWarning: The palette list has more values (13) than needed (12), w
hich may not be intended.
  sns.lineplot(data=df_median_end_common_grey,      palette=grey_palette,
dashes=False, legend=False, linewidth=1.5, alpha=0.8, ax=axes[1])
/var/folders/rw/j2_s688s5x9f9k_xg2xhfq980000gn/T/ipykernel_3008/2085485653.p
y:47: UserWarning: This figure includes Axes that are not compatible with ti
ght_layout, so results might be incorrect.
  plt.tight_layout()
```



4. Comparativa rápida CDMX vs paper USA

Resumen estadístico de los resultados CDMX para comparar con los valores reportados en Mauro et al. (2025) Tabla 1 / Fig. 2.

In [20]:
```
print("=" * 55)
print("  RESUMEN CDMX — G_bin y G_net por p_H")
print("=" * 55)
print(f"{'p_H':>6}  {'G_bin med':>10}  {'G_bin std':>10}  {'G_net med':>10}
print("-" * 55)
for p_g in sorted(pg2frac_chi.keys()):
    bin_vals = np.array(pg2frac_chi[p_g]) * 100
    net_vals = np.array(pg2frac_gamma[p_g]) * 100
    print(f"{p_g:>6.2f}  {np.median(bin_vals):>10.1f}  {np.std(bin_vals):>10
          f"{np.median(net_vals):>10.1f}  {np.std(net_vals):>10.1f}")
print("=" * 55)
print("Valores en % de celdas gentrificadas (de 25 totales)")
```

```
=======================================================
  RESUMEN CDMX — G_bin y G_net por p_H
=======================================================
   p_H   G_bin med   G_bin std   G_net med   G_net std
-------------------------------------------------------
  0.00         0.0         2.3        16.0         5.1
  0.01        16.0        10.3        36.0        10.7
  0.05        20.0        12.0        38.0        13.1
  0.10        28.0        22.7        48.0        21.6
  0.15        52.0        25.8        68.0        23.8
=======================================================
Valores en % de celdas gentrificadas (de 25 totales)
```

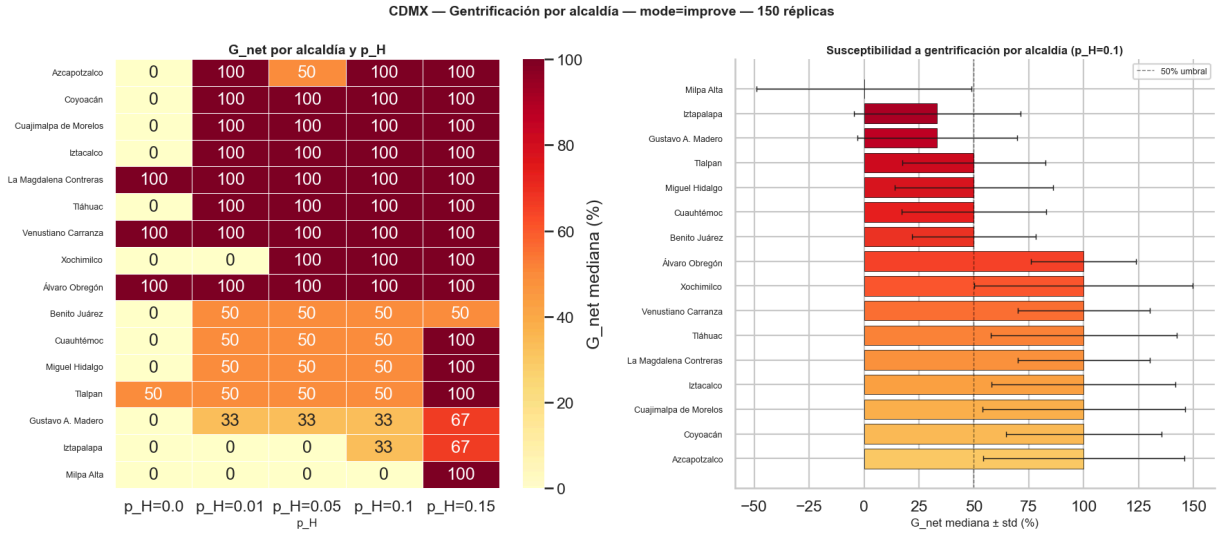
In [22]:
```
# ═══════════════════════════════════════════════════════════════════
# CELDA ADICIONAL: Mapeo celdas → alcaldías — CDMX
# Añadir después de la Sección 4 (Comparativa rápida) en batch_analysis_cdmx
# ═══════════════════════════════════════════════════════════════════

import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import defaultdict

# ── 1. Cargar mapa alcaldía → cell_ids ──────────────────────────────
with open("income/alcaldia_to_cells.json") as f:
    alcaldia_to_cells = json.load(f)

# Invertir: cell_id → alcaldía
cell_to_alcaldia = {}
for alc, cells in alcaldia_to_cells.items():
    for c in cells:
        cell_to_alcaldia[c] = alc

# ── 2. Mapear columnas de df_gamma a alcaldías ──────────────────────
# Las columnas son strings tipo '(0, 0)', '(1, 2)' etc.
# cell_id = row * width + col  (width=5)
def col_str_to_cell_id(col_str, width=5):
```

```
    row, col = eval(col_str)
    return row * width + col

# ── 3. Calcular G_bin y G_net por alcaldía sobre las 150 réplicas ──────────
p_g_list_map   = [0.0, 0.01, 0.05, 0.1, 0.15]
h_list_map     = [20]
delta_list_map = [15]

# alcaldia → p_g → lista de fracciones G_bin/G_net
alc2gbin  = defaultdict(lambda: defaultdict(list))
alc2gnet  = defaultdict(lambda: defaultdict(list))

for p_g in p_g_list_map:
    p_g_str   = f"pg_{p_g}_"
    for h in h_list_map:
        h_str = f"h_{h}_"
        for delta in delta_list_map:
            delta_str = f"delta_{delta}_"
            for file_name in os.listdir(intermediate_dir):
                if (p_g_str in file_name and h_str in file_name
                        and delta_str in file_name):
                    fpath = os.path.join(intermediate_dir, file_name)
                    df    = pd.read_csv(fpath, index_col=0)

                    # Agrupar columnas por alcaldía
                    alc_cols = defaultdict(list)
                    for col in df.columns:
                        cid = col_str_to_cell_id(col)
                        alc = cell_to_alcaldia.get(cid, "Desconocida")
                        alc_cols[alc].append(col)

                    if "net_avg_prod" in file_name:
                        for alc, cols in alc_cols.items():
                            sub = df[cols].fillna(0)
                            gent = sum(
                                len(find_peaks_custom(sub[c].reset_index(drc
                                for c in cols
                            )
                            alc2gnet[alc][p_g].append(gent / len(cols))

                    if "chi_hat" in file_name:
                        for alc, cols in alc_cols.items():
                            sub = df[cols].fillna(0)
                            gent = sum(
                                len(find_shifts(sub[c].reset_index(drop=True
                                for c in cols
                            )
                            alc2gbin[alc][p_g].append(gent / len(cols))

print("Alcaldías procesadas:", sorted(alc2gbin.keys()))

# ── 4. Tabla resumen por alcaldía (p_H=0.1, el más informativo) ────────────
p_g_ref = 0.1

rows = []
for alc in sorted(alc2gbin.keys()):
```

```
    bin_vals = np.array(alc2gbin[alc].get(p_g_ref, [0])) * 100
    net_vals = np.array(alc2gnet[alc].get(p_g_ref, [0])) * 100
    n_cells  = len(alcaldia_to_cells.get(alc, []))
    rows.append({
        "Alcaldía":     alc,
        "Celdas":       n_cells,
        "G_bin med (%)": round(np.median(bin_vals), 1),
        "G_bin std (%)": round(np.std(bin_vals), 1),
        "G_net med (%)": round(np.median(net_vals), 1),
        "G_net std (%)": round(np.std(net_vals), 1),
    })

df_alc = pd.DataFrame(rows).sort_values("G_net med (%)", ascending=False)
print(f"\n── G_bin y G_net por alcaldía — p_H={p_g_ref} ──")
print(df_alc.to_string(index=False))

# ── 5. Heatmap: susceptibilidad por alcaldía y p_H ──────────────────────
alcaldias_ord = df_alc["Alcaldía"].tolist()

# Matriz G_net mediana (alcaldía × p_H)
mat_net = np.array([
    [np.median(alc2gnet[alc].get(pg, [0])) * 100 for pg in p_g_list_map]
    for alc in alcaldias_ord
])

fig, axes = plt.subplots(1, 2, figsize=(18, 8))

sns.heatmap(
    mat_net,
    ax=axes[0],
    xticklabels=[f"p_H={p}" for p in p_g_list_map],
    yticklabels=alcaldias_ord,
    annot=True, fmt=".0f", cmap="YlOrRd",
    linewidths=0.5, cbar_kws={"label": "G_net mediana (%)"},
    vmin=0, vmax=100
)
axes[0].set_title("G_net por alcaldía y p_H", fontsize=14, fontweight="bold"
axes[0].set_xlabel("p_H", fontsize=12)
axes[0].tick_params(axis="y", labelsize=9)

# Barplot G_net en p_H=0.1 con error (std)
bin_meds = [np.median(alc2gnet[alc].get(p_g_ref, [0])) * 100 for alc in alca
bin_stds = [np.std(alc2gnet[alc].get(p_g_ref, [0])) * 100   for alc in alcal
colors   = plt.cm.YlOrRd(np.linspace(0.3, 0.95, len(alcaldias_ord)))

axes[1].barh(alcaldias_ord, bin_meds, xerr=bin_stds,
             color=colors, edgecolor="black", linewidth=0.5,
             error_kw={"elinewidth": 1, "capsize": 3})
axes[1].set_xlabel("G_net mediana ± std (%)", fontsize=12)
axes[1].set_title(f"Susceptibilidad a gentrificación por alcaldía (p_H={p_g_
                  fontsize=12, fontweight="bold")
axes[1].axvline(50, color="black", linestyle="--", linewidth=1, alpha=0.5,
                label="50% umbral")
axes[1].legend(fontsize=9)
axes[1].tick_params(axis="y", labelsize=9)
sns.despine(ax=axes[1])
```

```
plt.suptitle("CDMX — Gentrificación por alcaldía — mode=improve — 150 réplic
             fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig(
    f"out/plots/{starting_deployment}/{num_agents}/cdmx_gentrif_por_alcaldia
    bbox_inches="tight"
)
plt.show()

# ── 6. Ranking final legible ──────────────────────────────────────────
print("\n── RANKING: Alcaldías más susceptibles a gentrificación (G_net, p_H
print(df_alc[["Alcaldía", "Celdas", "G_net med (%)", "G_net std (%)"]].to_st
```

```
Alcaldías procesadas: ['Azcapotzalco', 'Benito Juárez', 'Coyoacán', 'Cuajima
lpa de Morelos', 'Cuauhtémoc', 'Gustavo A. Madero', 'Iztacalco', 'Iztapalap
a', 'La Magdalena Contreras', 'Miguel Hidalgo', 'Milpa Alta', 'Tlalpan', 'Tl
áhuac', 'Venustiano Carranza', 'Xochimilco', 'Álvaro Obregón']

── G_bin y G_net por alcaldía — p_H=0.1 ──
              Alcaldía  Celdas  G_bin med (%)  G_bin std (%)  G_net med (%)
G_net std (%)
          Azcapotzalco       1          100.0           48.0          100.0
45.8
              Coyoacán       1          100.0           39.0          100.0
35.4
 Cuajimalpa de Morelos       1          100.0           47.6          100.0
46.1
             Iztacalco       1          100.0           44.6          100.0
41.9
La Magdalena Contreras       1          100.0           26.1          100.0
30.0
               Tláhuac       1          100.0           43.1          100.0
42.3
   Venustiano Carranza       1          100.0           23.7          100.0
30.0
            Xochimilco       1            0.0           49.7          100.0
49.7
        Álvaro Obregón       2          100.0           15.8          100.0
23.9
         Benito Juárez       2           50.0           21.1           50.0
28.1
            Cuauhtémoc       2          100.0           19.2           50.0
32.8
        Miguel Hidalgo       2          100.0           20.7           50.0
35.9
               Tlalpan       2          100.0           23.3           50.0
32.7
     Gustavo A. Madero       3           66.7           16.0           33.3
36.3
            Iztapalapa       3          100.0            0.0           33.3
37.9
            Milpa Alta       1          100.0            0.0            0.0
48.8
```




```
── RANKING: Alcaldías más susceptibles a gentrificación (G_net, p_H=0.1) ──
                  Alcaldía  Celdas  G_net med (%)  G_net std (%)
              Azcapotzalco       1          100.0           45.8
                 Coyoacán        1          100.0           35.4
  Cuajimalpa de Morelos      1          100.0           46.1
                 Iztacalco       1          100.0           41.9
  La Magdalena Contreras     1          100.0           30.0
                 Tláhuac         1          100.0           42.3
    Venustiano Carranza      1          100.0           30.0
                 Xochimilco      1          100.0           49.7
            Álvaro Obregón       2          100.0           23.9
             Benito Juárez       2           50.0           28.1
                Cuauhtémoc       2           50.0           32.8
            Miguel Hidalgo       2           50.0           35.9
                   Tlalpan       2           50.0           32.7
          Gustavo A. Madero      3           33.3           36.3
                Iztapalapa       3           33.3           37.9
                Milpa Alta       1            0.0           48.8
```

In [ ]:

In [ ]: